

Name: Santiago, Jalene	Date Performed: March 2, 2026
Course/Section: CPE212 - 31S1	Date Submitted: March 2, 2026
Instructor: Engr. Robin E. Valenzuela	Semester and SY: 2nd Sem - 25-26
Activity 7: Managing Files and Creating Roles in Ansible	
1. Objectives: 1.1 Manage files in remote servers 1.2 Implement roles in ansible	
2. Discussion: In this activity, we look at the concept of copying a file to a server. We are going to create a file into our git repository and use Ansible to grab that file and put it into a particular place so that we could do things like customize a default website, or maybe install a default configuration file. We will also implement roles to consolidate plays.	
Task 1: Create a file and copy it to remote servers - Using the previous directory we created, create a directory, and named it "files." Create a file inside that directory and name it "default_site.html." Edit the file and put basic HTML syntax. Any content will do, as long as it will display text later. Save the file and exit. - Edit the site.yml file and just below the web_servers play, create a new file to copy the default html file for site: - name: copy default html file for site - tags: apache, apache2, httpd - copy: - src: default_site.html - dest: /var/www/html/index.html - owner: root - group: root - mode: 0644 - Run the playbook site.yml. Describe the changes. - Go to the remote servers (web_servers) listed in your inventory. Use cat command to check if the index.html is the same as the local repository file (default_site.html). Do both for Ubuntu and CentOS servers. On the CentOS server, go to the browser and type its IP address. Describe the output. - Sync your local repository with GitHub and describe the changes.	

TASK 1 OUTPUT

```
• jalene@workstation:~/CPE212_Santiago$ cd HOA-7
• jalene@workstation:~/CPE212_Santiago/HOA-7$ ls
ansible.cfg      files            site_playbook.yml
default_site.html inventory.ini    site.yml
```

```
! site.yml u x
HOA-7 > ! site.yml
21 - hosts: web_servers
24   tasks:
44     - name: Start httpd (CentOS)
46       service:
49         when: ansible_distribution == "CentOS"
50
51     - name: copy default html file for site
52       tags: apache, apache2, httpd
53       copy:
54         src: files/default_site.html
55         dest: /var/www/html/index.html
56         owner: root
57         group: root
58         mode: 0644
59
```

```
TASK [copy default html file for site] *****
changed: [server2]
```

```
jalene@workstation:~/CPE212_Santiago/HOA-7$ ansible-playbook site.yml -K
BECOME password:

PLAY [all] *****

TASK [Gathering Facts] *****
ok: [server2]
ok: [server1]
ok: [server3]

TASK [Install Updates (CentOS)] *****
skipping: [server1]
skipping: [server2]
skipping: [server3]

TASK [Install Updates (Ubuntu)] *****
[WARNING]: The value "True" (type bool) was converted to "'True'" (type string). If this does not look like what you
expect, quote the entire value to ensure it does not change.
ok: [server2]
ok: [server3]
ok: [server1]

PLAY [web_servers] *****

TASK [Gathering Facts] *****
ok: [server2]

TASK [Install Apache and PHP for Ubuntu Servers] *****
ok: [server2]

TASK [Install Apache and PHP for CentOS Servers] *****
skipping: [server2]
```

```

TASK [copy default html file for site] *****
ok: [server2]

PLAY [db_servers] *****

TASK [Gathering Facts] *****
ok: [server1]

TASK [Install Mariadb Package(CentOS)] *****
skipping: [server1]

TASK [Mariadb Restarting/Enabling] *****
changed: [server1]

TASK [Install Mariadb Package(Ubuntu)] *****
ok: [server1]

TASK [Mariadb Restarting/Enabling] *****
changed: [server1]

PLAY [file_servers] *****

TASK [Gathering Facts] *****
ok: [server3]

TASK [Install samba package] *****
changed: [server3]

PLAY RECAP *****
server1      : ok=6    changed=2    unreachable=0    failed=0    skipped=2    rescued=0    ignored=0
server2      : ok=5    changed=0    unreachable=0    failed=0    skipped=3    rescued=0    ignored=0
server3      : ok=4    changed=1    unreachable=0    failed=0    skipped=1    rescued=0    ignored=0

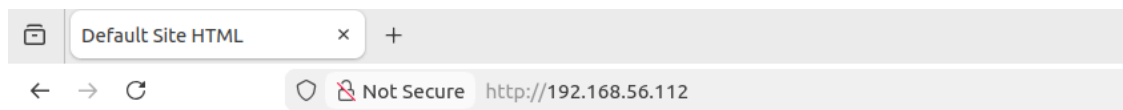
```

```

Last login: Mon Mar  2 22:03:44 2026 from 192.168.56.110
jalene@server2:~$ cat /var/www/html/index.html
<html>
<head>
<title>Default Site HTML</title>
</head>

<body>
<h1>Hands-on Activity 7</h1>
<p>This site was deployed using Ansible.</p>
</body>
</html>
jalene@server2:~$

```



Hands-on Activity 7

This site was deployed using Ansible.

Task 1 started with creating another directory inside the repository. I was tasked to create the default_site.html and put a basic HTML syntax. After creating the HTML file, I also edited the site.yml to allow the playbook to copy default_site.html from local repo to the given destination directory of the web server. The playbook was successful and the contents of the file were verified using the cat command on the remote server. Accessing the web server's IP address in a browser displayed the custom HTML page, confirming that the file was correctly copied and deployed.

Task 2: Download a file and extract it to a remote server

1. Edit the site.yml. Just before the web_servers play, create a new play:

- hosts: workstations
become: true
tasks:
 - name: install unzip
package:
 - name: unzip
 - name: install terraform
unarchive:

src:

https://releases.hashicorp.com/terraform/0.12.28/terraform_0.12.28_linux_amd64.zip

dest: /usr/local/bin
remote_src: yes
mode: 0755
owner: root
group: root

2. Edit the inventory file and add workstations group. Add any Ubuntu remote server. Make sure to remember the IP address.

3. Run the playbook. Describe the output.

4. On the Ubuntu remote workstation, type terraform to verify installation of terraform. Describe the output.

TASK 2 OUTPUTS

```
- hosts: workstations
  become: true
  tasks:
    - name: install unzip
      package:
        name: unzip
        state: present
    - name: install terraform
      unarchive:
        src: https://releases.hashicorp.com/terraform/0.12.28/terraform_0.12.28_linux_amd64.zip
        dest: /usr/local/bin
        remote_src: yes
        mode: 0755
        owner: root
        group: root
```

```
PLAY [workstations] *****

TASK [Gathering Facts] *****
ok: [server2]

TASK [install unzip] *****
ok: [server2]

TASK [install terraform] *****
changed: [server2]
```

```
jalene@server2:~$ terraform
Usage: terraform [-version] [-help] <command> [args]

The available commands for execution are listed below.
The most common, useful commands are shown first, followed by
less common or more advanced commands. If you're just getting
started with Terraform, stick with the common commands. For the
other commands, please read the help and docs before usage.

Common commands:
  apply          Builds or changes infrastructure
  console        Interactive console for Terraform interpolations
  destroy        Destroy Terraform-managed infrastructure
  env            Workspace management
  fmt            Rewrites config files to canonical format
```

Task 2 focuses on installing the unzip package and downloading the Terraform file from the internet. I only assign server2 for web_server, which explains the output only showing server2. The playbook extracted the Terraform so it can run on the server. After running the playbook, I performed ssh on server2 and verified the installation by typing 'terraform', and it showed the Terraform command menu, which verifies the installation.

Task 3: Create roles

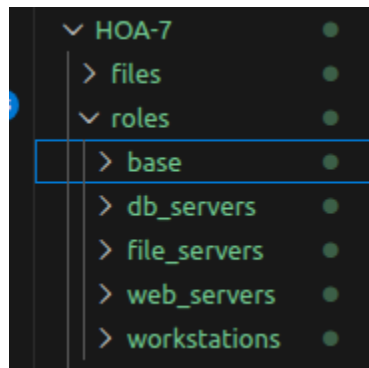
1. Edit the site.yml. Configure roles as follows: (make sure to create a copy of the old site.yml file because you will be copying the specific plays for all groups)

```
---
- hosts: all
  become: true
  pre_tasks:
    - name: update repository index (CentOS)
      tags: always
      dnf:
        update_cache: yes
        changed_when: false
        when: ansible_distribution == "CentOS"
    - name: install updates (Ubuntu)
      tags: always
      apt:
        update_cache: yes
        changed_when: false
        when: ansible_distribution == "Ubuntu"
- hosts: all
  become: true
  roles:
    - base
- hosts: workstations
  become: true
  roles:
    - workstations
- hosts: web_servers
  become: true
  roles:
    - web_servers
- hosts: db_servers
  become: true
  roles:
    - db_servers
- hosts: file_servers
  become: true
  roles:
    - file_servers
```

Save the file and exit.

2. Under the same directory, create a new directory and name it roles. Enter the roles directory and create new directories: base, web_servers, file_servers, db_servers and workstations. For each directory, create a directory and name it tasks.
3. Go to tasks for all directory and create a file. Name it main.yml. In each of the tasks for all directories, copy and paste the code from the old site.yml file. Show all contents of main.yml files for all tasks.
4. Run the site.yml playbook and describe the output.

TASK 3 OUTPUTS



```

EXPLORER
CPE212_SANTIAGO
  HOA-6
  HOA-7
    files
    default_site.yml
    old_site.yml
    roles
      base
      db_servers
      file_servers
      web_servers
      workstations
    ansible.cfg
    inventory.ini
    site.yml
    ansible.cfg
    install_apache.yml
    inventory.ini

HOA-7 > ! site.yml
1 ---
2 - hosts: all
3   become: true
4   roles:
5     - base
6
7 - hosts: web_servers
8   become: true
9   roles:
10    - web_servers
11
12 - hosts: db_servers
13   become: true
14   roles:
15    - db_servers
16
17 - hosts: file_servers
18   become: true
19   roles:
20    - file_servers
21
22 - hosts: workstations
23   become: true
24   roles:
25    - workstations
  
```

```

PLAY RECAP *****
server1      : ok=6    changed=2    unreachable=0    failed=0    skipped=2    rescued=0    ignored=0
server2      : ok=8    changed=0    unreachable=0    failed=0    skipped=3    rescued=0    ignored=0
server3      : ok=4    changed=0    unreachable=0    failed=0    skipped=1    rescued=0    ignored=0
  
```

Task 3 implements the use of roles in playbook instead of monolithic, the playbook was modified to use roles instead of placing all tasks in a single file. Separate roles were created for base configuration, web servers, database servers, file servers, and workstations. Each role contains a

tasks/main.yml file that holds the tasks for that server group. Running the playbook executed the tasks through their respective roles, making the playbook more organized and easier to manage.

Reflections:

Answer the following:

1. What is the importance of creating roles?

Instead of using a monolithic playbook, creating roles organizes Ansible playbooks by separating tasks into structured and reusable components. This makes the playbooks a lot easier to manage, maintain, and reuse across different servers or projects.

2. What is the importance of managing files?

Managing files allows administrators to automatically copy, update, and configure important files on remote servers. Managing files makes sure that systems have clean and consistent configurations and reduces the need for manual setup.